becomes an extremely tedious and error-prone process.

A scalable solution is to create a *dotenv* file per environment for development, staging and production, as shown in Figure 4. We overwrite all the values and build a system that can just ship this file to the deployment infrastructure.

The whole process is now automated, and you just have to change the file and automatically sync to any deployment system that is being used. Once you have separate *dotenv* files, you can simply modify the code to load from whichever stage of production.

After getting the configuration, you can just call the MongoDB connection with config variables. The configuration should be written separately for every single environment and committed to the GitHub repository. Then, as the code is deployed using the continuous integration/continuous delivery (CI/CD) pipeline, it automatically loads the environment variables and runs, with no manual communication needed between the DevOps and engineering teams.

The *dotenv* file should be committed to Git and the environment variable must be set once in the deployment infrastructure manually. For the rest of the future deployments, the code will automatically run by passing on the variables.

The problem here is the need to put all these credentials for the production database in the *dotenv* production file and also commit it to Git. This is a huge security risk as you are sharing secret credentials with everyone in the project. There is the risk of any hacker getting access to the Git repository, even if you are working alone. Hence, fundamentally, never store credentials in plain text, or send them over Slack or any other messaging medium.

One solution here is to use a secrets manager, which will encrypt secrets and also have an access control built into it. So when the code asks for the encrypted key, the secrets manager automatically decrypts it and returns to the value. This allows you to manage permissions to people having access to the secrets, making the code more secure.

There are some secret managers that even provide active support for rotating secrets. For example, if the database credentials are stored in a secrets manager, the latter can automatically rotate these credentials every six months. So all the apps using the credentials from the secrets manager will start receiving the new ones after six months, and the database will also move to the new credentials. AWS Parameter Store, AWS Secrets Manager and cloud-agnostic solutions like Hashicorp Vault, GCP Secrets Engine and Azure Key Vault are a few examples of secrets managers that also make an app look simple.

You can leave *dotenv* files and move only sensitive parameters to a secrets manager. The code can be changed to read these variables from the two sources.

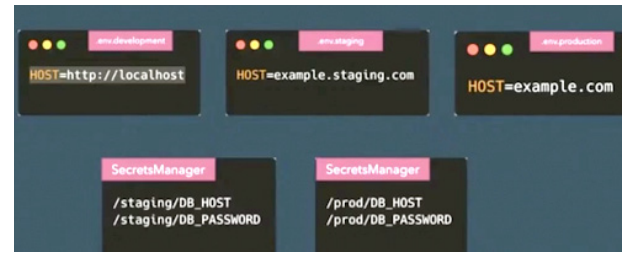

Figure 6: MongoDB credentials move to a secrets manager

```
config = loadFromEnv(.env.{stage})
if (production) {
  dbSecrets = getProdSecretsUsingApi()
} else if(staging) {
  dbSecrets = getStagingSecretsUsingApi()
} else {
  dbSecrets = getDevSecretsUsingApi()
}

server = Server()
mongoDB = new MongoDb(
  dbSecrets.DB_USER,
  dbSecrets.DB_PASSWORD,
  dbSecrets.DB_HOST,
  dbSecrets.DB_PORT,
  dbSecrets.NAME
))

server.get('/users', function() {
  mongoDB.getUsers()
})

server.start(port=config.PORT)
```

Figure 7: App code with secrets manager is not simple anymore

The code will now load things first from the *dotenv* file and then contact secrets manager API to get the actual value of the secrets. The API may vary accordingly, since the secrets are different for staging, production or development.

The code gets the secrets inside the application, and depending on which secrets manager is being used, the corresponding library is imported. This may not be a good solution, as there can be a lot of duplication of code for every single application inside the company. Abstracting it out as a library can make the code less feasible, and adding/deleting environments dynamically can be tedious.

Managing secrets via SecretsFoundry

Since there are major issues in moving the code to fetch secrets from inside the application and integrating it with multiple secrets managers, SecretsFoundry, a cloud-agnostic solution to make secrets management simple and secure, is of help. It figures out a solution to load all the secrets, passes it back to your application in a secure way, and also works on all platforms with different secrets managers.

Your code should now have a flow from *dotenv* files (production, staging or development) for each of the environments and then commit to Git. You are actually not